

CS 223

Week 3 Lecture 1



Agenda

Announcements

Review (Stack / Heap Diagrams, Malloc, Arrays)

2D Arrays

Pointer operators: *, &

Pass by pointer for non-array types

Struct pointers

Pointer safety

NULL pointers

Return values

Announcements

Lab this week: practice quiz!

HW#2 will be released Friday, Sept 18 and due Thursday Sept 24 at midnight

Reminder:

- My office hours are Mon 11am-12pm and Thursday 2:30pm-3:30pm (and by appointment) in Park 204
- Linh's are Fridays 3pm-5pm in Park 231

How can malloc fail?

Exhaustion of physical or virtual memory

Heap fragmentation (lack of contiguous space)

Process resource limits

Integer overflows (requesting an impossibly large block)

Zero-size allocation (malloc(0))

Atoi and atof

atoi → string to integer

atof → string to float

```
char str[] = "12345";
```

```
int num;
```

```
// Convert string to integer
```

```
num = atoi(str);
```

Review from last time

Stack / Heap Diagrams, Malloc, Arrays

Exercise: How big is `b` in memory?

```
struct bar {  
    char str[4];  
    float x,y,z;  
};  
int main() {  
    struct bar b;  
    b.str[0] = 'l';  
    b.str[1] = 'o';  
    b.str[2] = 'l';  
    b.str[3] = '\\0';  
    b.x = 0.0f;  
    b.y = 1.0f;  
    b.z = 2.0f;  
}
```

Pointer or not?

Pointer syntax: which of these variables are pointers?

- `char test[3];`
- `char* test;`
- `char test;`
- `const char* test;`
- `char** test;`
- `char* test[5];`
- `char test[];`
- `int test;`
- `int test[1000];`
- `int* test;`

Malloc and Free

malloc = “memory allocation”

If **malloc** is successful

Returns address to beginning of memory block
(e.g. the **base address**)

Else

malloc returns NULL

free returns the memory to the heap where it can be reused

Example: Malloc and Free

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr;
    char *c_arr;

    // allocate an array of 20 ints on the heap:
    arr = malloc(sizeof(int) * 20);

    // allocate an array of 10 chars on the heap:
    c_arr = malloc(sizeof(char) * 10);
    free(arr);
    free(c_arr);
    return 0;
}
```

Rules of Thumb:

- Pointer types should match the data type
- Use `sizeof()` to get the data type size
- For every `malloc`, make sure you have a corresponding `free`

Example: Malloc and Free

Draw the stack diagram for this program.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr;

    arr = malloc(sizeof(int) * 20);
    arr[1] = 10;

    arr = malloc(sizeof(int) * 10);
    free(arr);
    return 0;
}
```

Example: malloc and free

This code crashes. Why?

```
int *arr = NULL;
char *c_arr = NULL;

arr = malloc(sizeof(int) * 20);
c_arr = malloc(sizeof(char) * 10);

free(arr);
arr[0] = 2;
c_arr[0] = 'd';

free(c_arr);
```

Example: malloc and free

This code crashes. Why?

```
int *arr = NULL;
char *c_arr = NULL;

arr = malloc(sizeof(int) * 20);
c_arr = malloc(sizeof(char) * 10);

free(arr);
free(c_arr);
free(arr);
```

Example: Passing simple types to functions

```
int init(int a, int size) {  
    float value = a * size;  
    return value;  
}  
  
int main() {  
    int b = 10;  
    int x = 3;  
    int temp = init(b, x);  
    // draw stack here  
  
    printf("temp: %d\n", temp);  
}
```

Example: Passing arrays to functions

```
void init_array(int *arr, int size) {  
    for (int i = 0; i < size; i++) {  
        arr[i] = i;  
    }  
}
```

```
int main() {  
    int arr1[4];  
  
    init_array(arr1, 4);  
    // draw stack here  
}
```

Array elements are contiguous in memory

Each element in an array of type T is sizeof(T) apart in memory,

e.g. `int arr[10]` stores 10 integers consecutively in memory

2D arrays

A 2D array is an array of arrays

```
int matrix[50][100];
```

Each row is a pointer to an array of columns

2D arrays as contiguous blocks

```
int matrix[50][100];
```

		columns					
matrix:		0	1	2	3	...	99
rows	0					...	
	1					...	
	2				8	...	
	3					...	
	4					...	
	⋮	⋮	⋮	⋮	⋮	⋮	⋮
	49					...	13

```
matrix[2][3] = 8;  
matrix[49][99] = 13;
```

Dynamic 2D arrays

Two approaches:

Use single call to malloc (more efficient)

Use multiple calls to malloc: one to create the 2D array followed by a malloc to allocate each row

2D arrays as contiguous blocks

Mental model is to think of the 2D array as a grid but the reality is that memory is stored in **row-major order** as a $\text{NROWS} * \text{NCOLS}$ array

```
// location = row * NCOLS + col
```

```
int arr[3][4];
```

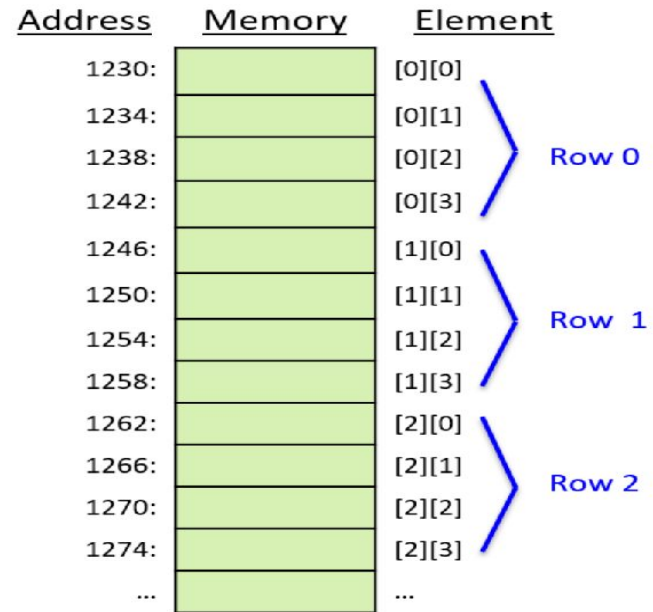


Figure 21. The layout of a two-dimensional array in row-major order.

Dynamic 2D array: Method 1

“Flat 2D array”

```
int *two_d_array;
```

```
two_d_array = malloc(sizeof(int) * (3*4));
```

main:

two_d_array:

addr in heap
(9230)

Stack

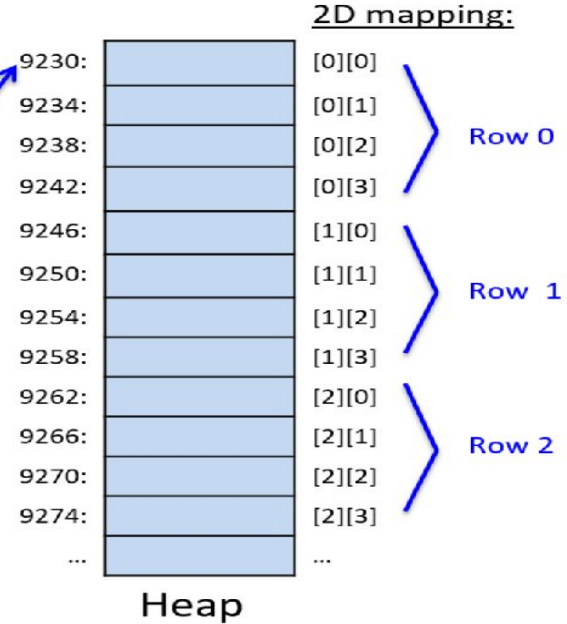


Figure 22. The results of allocating a 2D array with a single call to malloc.

How to index?

Flat 2D array: Computing indices

Suppose we use a flat array to store a 4x4 matrix. What is the index for element (2, 3)?

Suppose we use a flat array to store a 3x10 matrix. What is the index for element (2, 3)?

Suppose we use a flat array to store a 10x3 matrix. What is the index for element (4, 0)?

Flat 2D array: Computing row, col given an index

What is the general formula for converting from (i,j) to an index for an $N \times M$ matrix stored as a flat array?

What is the general formula for converting from a 1D index to a 2D index (i,j) for an $N \times M$ matrix stored as a flat array?

Dynamic 2D array: Method 2

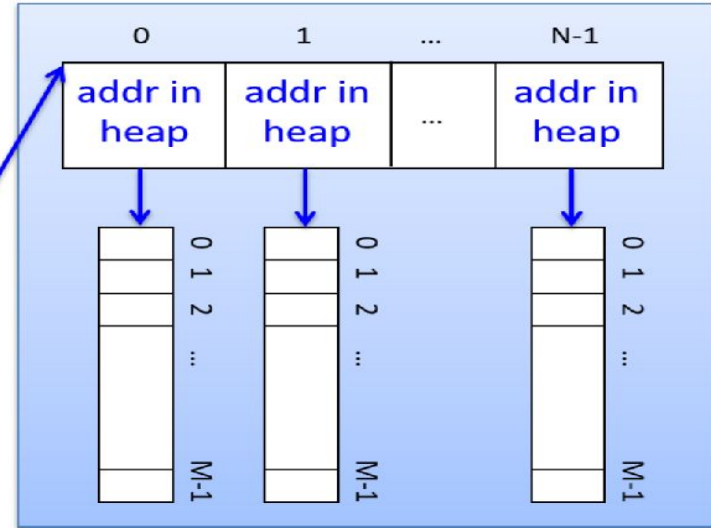
“Array of arrays”

```
int **two_d_array;  
  
two_d_array = malloc(sizeof(int *) * N);  
for (i=0; i < N; i++) {  
    two_d_array[i] = malloc(sizeof(int) * M);  
}
```

main:

two_d_array: addr in heap

Stack



Heap

Figure 23. The arrangement of memory after allocating a 2D array with $N+1$ malloc calls.

How to index?

Exercise: 2D array using flat 2D array

Sketch a program, **ask_matrix_flat.c**, that creates and frees a 3X4 matrix of integers.

Exercise: 2D arrays using “array of arrays”

Sketch a program, **ask_matrix_aa.c**, that creates and frees a 3X4 matrix.

Review

What is the function, or execution, stack?

What is a pointer?

What is the difference between static and dynamic allocation of memory?

What are two ways we can allocation 2D arrays dynamically?

Why use pointers?

- Change variables in functions so they persist when the function returns
- Dynamically allocate space for data
- Pass variables without copying
- Create linked data structures
- Interpret memory in different ways

Pointer operators: &, *

& - **address** operator, gives the address (its location in memory) of a variable

* - **dereference** operator, refers to the value of a pointer

```
#include <stdio.h>
```

```
int main() {  
    char c1 = 'a';  
    char* ptr;  
    ptr = &c1;  
    char c2 = *ptr;  
    printf("%c %c\n", c1, c2);  
    return 0;  
}
```

Pointer syntax

`<data_type>*`: declares a pointer type

`*<variable>`: Dereference a pointer to get the corresponding value at its address

`&<variable>`: address operator, gets the address where this variable is stored

Pointers

Be careful: * has two meanings!

```
void pass_by_pointer(int* v) {  
    *v = 10;  
}
```

Exercise: Draw a stack diagram for this function

```
int main() {  
    int *ptr1, *ptr2, x, y;  
    x = 8;  
    ptr2 = &x;  
    ptr1 = NULL;  
    *ptr2 = 10;  
    y = *ptr2 + 3;  
    ptr1 = ptr2;  
    *ptr1 = 100;  
    ptr1 = &y;  
    *ptr1 = 80;  
    // draw stack here  
}
```

Pointers and Functions Revisited

Two choices for passing data

- Pass by value: copies values into parameters of a function (simple types)
- Pass by pointer: Copies the address of data. Changes to the data persist when the function returns (all arrays and pointers)

Example: pass by pointer for non-array types

```
void pass_by_value(int v) {  
    v = 10;  
}  
  
void pass_by_pointer(int* v) {  
    *v = 10;  
}  
  
int main() {  
    int changeme = 0;  
    pass_by_value(changeme);  
    printf("The value is: %d\n", changeme);  
  
    pass_by_pointer(&changeme);  
    printf("The value is: %d\n", changeme);  
    return 0;  
}
```

What is the syntax for pass by pointer?

Pass by pointer: draw stack

```
void pass_by_value(int v) {  
    v = 10;  
}  
  
void pass_by_pointer(int* v) {  
    *v = 10;  
}  
  
int main() {  
    int changeme = 0;  
    pass_by_value(changeme);  
    printf("The value is: %d\n", changeme);  
  
    pass_by_pointer(&changeme);  
    printf("The value is: %d\n", changeme);  
    return 0;  
}
```

Example: Pass by pointer

```
#include <stdio.h>

void inc(int* i) {
    int local = *i;
    local = local + 1;
    *i = local;
}

int main() {
    int i = 5;
    printf("Before: %d\n", i);
    inc(&i);
    printf("After: %d\n", i);
    return 0;
}
```

Exercise: rand_point.c

Write a program that uses pass by pointer to set the values of x, y, and z to random positive integers in range [0, 10)

Struct pointers

Works like any other data type!

BUT use -> to access data inside the struct

Rule:

Use -> for struct **pointers**

Use . For struct **values**

Recall: struct pass by value

```
void changeName(struct studentT s, char* newName) {  
    if (newName == NULL) {  
        return;  
    }  
    strcpy(s.name, newName);  
}
```

```
void changeGpa(struct studentT s, float newGpa) {  
    s.gpa = newGpa;  
}
```

What is the output of this program?
Draw the stack diagram.

```
int main() {  
    struct studentT student1;  
  
    strcpy(student1.name, "Ruth");  
    student1.gpa = 3.5;  
  
    print("ORIGINAL", student1);  
    changeName(student1, "Kwame");  
    print("AFTER CHANGE NAME", student1);  
  
    changeGpa(student1, 3.7);  
    print("AFTER CHANGE GPA", student1);  
  
    return 0;  
}
```

```
struct studentT {  
    char name[8];  
    float gpa;  
};
```

struct pass by pointer

```
void changeName(struct studentT* s, char* newName) {  
    if (newName == NULL) {  
        return;  
    }  
    strcpy(s->name, newName);  
    // DRAW STACK HERE  
}  
  
void changeGpa(struct studentT* s, float newGpa) {  
    s->gpa = newGpa;  
}
```

What syntax is different from the previous program?
What is the output of this program?

```
int main() {  
    struct studentT student1;  
  
    strcpy(student1.name, "Ruth");  
    student1.gpa = 3.5;  
  
    print("ORIGINAL", student1);  
    changeName(&student1, "Kwame");  
    print("AFTER CHANGE NAME", student1);  
  
    changeGpa(&student1, 3.7);  
    print("AFTER CHANGE GPA", student1);  
  
    return 0;  
}
```

Compiles or not?

```
void changeName(struct studentT s, char* newName) {  
    if (newName == NULL) {  
        return;  
    }  
    strcpy(s->name, newName);  
}
```

Compiles or not?

```
void changeName(struct studentT* s, char* newName) {  
    if (newName == NULL) {  
        return;  
    }  
    strcpy(s->name, newName);  
}
```

Compiles or not?

```
void changeName(struct student* s, float value) {  
    s->gpa = value;  
}
```

```
void changeName(struct student* s, float value) {  
    s.gpa = value;  
}
```

```
void changeName(struct student* s, float value) {  
    s->gpa = &value;  
}
```

Pointer safety

Using an invalid pointer will crash your program

Common pointer mistakes

- Using an uninitialized pointer
 - Solution: initialize pointers to NULL
- Dereferencing a NULL pointer
 - Solution: Check for NULL before using
- Freeing memory that has already been freed
 - Solution: Reset memory to NULL after free
- Dereferencing a pointer that refers to deleted or invalid memory
- Casting a pointer to a type that doesn't match the data (more later)

Use **GDB** and **Valgrind** to help you find memory problems in your code

Example: Checking for NULL

```
int main() {  
    int* value = NULL;  
    if (value != NULL) {  
        printf("value is %d\n", *value);  
    }  
  
    int a = 4;  
    value = &a;  
  
    if (value != NULL) {  
        printf("value is %d\n", *value);  
    }  
}
```

Return values

When we return a value, the *value* is copied

When we return a pointer, the *address* is copied

If the address refers to memory that is destroyed when the function stack pops, the address will be invalid

NEVER return an address to a local variable

Return value gotchas: Safe or not?

```
char* code(int v) {  
    char msg[16];  
    if (v == 0) strcpy(msg, "val0");  
    else if (v == 1) strcpy(msg, "val1");  
    else if (v == 2) strcpy(msg, "val2");  
    return msg;  
}  
  
int main() {  
    int val = 1;  
    char* printme = code(val);  
    printf("%s\n", printme);  
    return 0;  
}
```

Draw the function stack/heap.
Is this return value safe or not?

Return value gotchas: Safe or not?

```
char* code(int v) {  
    if (v == 0) return "val0";  
    else if (v == 1) return "val1";  
    return "val2";  
}  
  
int main() {  
    int val = 1;  
    char* printme = code(val);  
    printf("%s\n", printme);  
    return 0;  
}
```

Draw the function stack/heap. Assume val = 1
Is this return value safe or not?

Return value gotchas: Safe or not?

```
char* code(int v) {  
    char* msg = malloc(sizeof(char) * 16);  
    if (v == 0) strcpy(msg, "val0");  
    else if (v == 1) strcpy(msg, "val1");  
    else if (v == 2) strcpy(msg, "val2");  
    return msg;  
}  
  
int main() {  
    int val = 1;  
    char* str = code(val);  
    printf("%s\n", str);  
    return 0;  
}
```

Draw the function stack/heap.
Is this return value safe or not?
What about memory leaks?

Return value gotchas: Safe or not?

```
void code(int v, char* msg) {  
    if (v == 0) strcpy(msg, "val0");  
    else if (v == 1) strcpy(msg, "val1");  
    else if (v == 2) strcpy(msg, "val2");  
}
```

```
int main() {  
    int val = 1;  
  
    char msg[16];  
    code(val, msg);  
    printf("%s\n", msg);  
    return 0;  
}
```